

Read/Write Locality and Page Replacement Cost: A Simulator Study in Systems Programming Suite

Zelu Zhang

SPersonal Project and Research

Abstract

Hit rate and page-fault counts are the usual yardsticks for page replacement, but they miss a real expense: evicting a page that has been written back to storage costs more than evicting a clean one. This paper takes the replacement core of Systems Programming Suite, a personal systems project, and asks how the read/write ratio, where writes fall, how many frames are resident, and how policies sample reference bits together shape faults, writebacks, and a writeback-weighted cost. We describe the suite and its virtual memory design, then measure the six policies the project already ships: FIFO, Random, Clock, NRU, Aging, and an approximate Working Set. No CPU cache hierarchy and no second cache layer are introduced; the experiments stay inside the page-residency mechanism the project actually implements. The frozen dataset holds 1,082 unique inputs, 31,780 unique core runs, and 90 virtual memory integration cases. The main study repeats each configuration over 20 inputs, with Random first averaged across five policy seeds per input. At 16 frames, 50% uniformly placed writes, and a writeback-to-load cost ratio of $\lambda = 5$, Aging costs 19.08%, 14.27%, and 18.33% less than Clock on the hotspot, scan, and hotspot-shift workloads. Hold the write ratio at 10% and move only where the writes land, and NRU swings from 46.81% cheaper than Clock (writes on hot pages) to 11.38% more expensive (writes on cold pages). The lesson is that neither the hit rate nor the overall write ratio settles the ranking on its own; what matters is how writes correlate with reuse, and how capacity and sampling interact with it. What we offer is a controlled, auditable measurement of code that already exists, not a new algorithm and not a claim about real hardware.

Keywords: virtual memory; page replacement; read/write ratio; write locality; dirty-page writeback; reproducibility

1 Introduction

A virtual memory system backs many pages with a small number of physical frames. A request for an absent page forces a load, and if the frame chosen to make room holds a modified page, the eviction also forces a writeback. Raising the hit rate and cutting writebacks are therefore related goals, but they are not the same goal, and a policy can do well on one while doing poorly on the other. Heterogeneous storage costs, a preference for evicting clean pages, and write-aware eviction histories have all been studied before [2, 4, 7, 9]. The question this project asks is narrower: what do these tradeoffs look like inside a simulator whose semantics you can read off the source?

Systems Programming Suite runs a single path from a configuration file down to native simulation events, and it lets you step through those events one at a time. The virtual memory module supplies the setting for this study, and its shared `MemoryMachine` is the object we measure. Two entry points drive it. The single-process memory interface accepts longer page traces than the VM instruction interface, but both call the same replacement core, so long

traces isolate the replacement decisions while the shorter VM cases exercise address spaces, page sources, and the process lifecycle. Keeping both on one core is deliberate: the measurement adapter is a view onto the project, not a second simulator standing in for it.

The paper addresses four research questions:

- RQ1: When the page request order is fixed, how do the write ratio and the placement of writes move the hit rate, the number of dirty evictions, and the total modeled cost?
- RQ2: Do the differences between policies survive changes in resident capacity and in the weight placed on writebacks?
- RQ3: How do scans and hotspot shifts change phase behavior, and how much do the results depend on sampling intervals and working-set windows?
- RQ4: How do warmup, the finite ends of a trace, and VM page lifecycles change the way cost should be read?

The work extends the project rather than replacing it. It adds a measurement interface built on the existing core, a pair of workload generators that pull write ratio apart from write placement, an audited dataset of raw counters and per-window statistics, and an analysis that stays tied to how the code behaves. It does not introduce a new policy, crown an overall winner, or benchmark the more advanced algorithms discussed as related work.

2 Systems Programming Suite

2.1 Project Architecture and Experimental Entry Points

The suite gathers several systems components under one roof: memory replacement, virtual memory, CPU scheduling, disk scheduling, linking, and compilation. Python handles configuration, dispatch between modules, and the browser and command-line front ends, while the C++17 engines emit inspectable state events. The compiler even carries a small language with an RV64IM execution path. Of all this, only the memory and virtual memory modules feed the replacement experiments here; the rest is context for where those modules sit.

The memory interface takes up to 64 frames and 4,096 page accesses. Each input names a page sequence, the indices at which writes occur, the policy, a Random seed, the reference-bit reset interval, and the Working Set window. The VM interface is smaller: up to 32 frames, 256 instructions, and eight processes of up to 64 virtual pages each. Both hand replacement to `core/memory.cpp`. The original interface keeps a full frame snapshot after every access; the research adapter links the same core but exports summary counters and per-window statistics, which is what batch experiments need.

2.2 Why the Study Uses One Resident Frame Pool

The resident frame pool is itself a cache: a request hits when its page is resident and faults when a valid page is not. Every mention of caching in this paper refers to that page-residency mechanism, not to CPU L1/L2/L3 caches. We add no general second cache and no per-policy cache. Bolting on another layer would rewire the hit paths, change how dirty pages propagate, and move the cost boundaries, which would introduce a new design variable before the existing policies are even understood. Cache-replacement research still bears on page management, but the knobs we turn here are limited to what the shared core already supports.

3 Virtual Memory Design

3.1 Address Spaces, Page Tables, and Global Replacement

The VM module splits a byte address into a virtual page number and an offset, with a default page size of 4,096 bytes. Once the page table maps a virtual page to frame f , the physical address is

$$a_{\text{phys}} = 4096f + \text{offset}. \quad (1)$$

Each process keeps a private page table, and the replacement core tells apart identically numbered virtual pages in different processes through a page key built from the process index and the virtual page number. There are no per-process frame quotas: every process draws from one physical frame pool under one global policy. What the multi-process cases test, then, is address-space separation and competition for a shared capacity, not scheduling time or context-switch latency, which the model does not represent.

3.2 Page Sources, Dirty State, and Process Exit

An anonymous page taken for the first time counts as a zero fill. Once it has acquired swap backing, a later fault on it counts as a swap-in. Evicting a dirty anonymous page is a swap-out; loading a file-mapped page is a file-in, and dirty-evicting one is a file-out. The simulator logs these events and the state changes behind them. It moves no real bytes, issues no storage requests, and does not model shared file-cache identity or block-level write amplification.

A valid write sets the dirty bit, and a read hit never clears it. When a process exits, its frames are released: anonymous data can simply be dropped, but dirty resident file-mapped pages are written out first. A write-protection failure can surface after a page is loaded, yet it does not dirty the page. The research VM cases issue only valid reads and writes, so protection faults and invalid-address exceptions stay out of the hit-rate comparisons. In the memory interface every configured page is valid, which means a fault always signals absence from the frame pool rather than an address error. One detail matters for later: when a memory trace ends, dirty pages left resident stay dirty, and the core does not drain them on its own.

3.3 Shared Access Semantics and Replacement Policies

Every access runs in the same order. First the policy does any periodic sampling it requires; then the core looks for the requested page. A miss takes a free frame if one exists, and otherwise selects a victim, charging the writeback counter when that victim is dirty. A freshly loaded page starts clean, at age zero, with timestamps set to the current access index. The core then sets the accessed frame's reference bit, and a write sets its dirty bit too. The clock hand starts at frame zero and steps to the frame just after each load; hits do not move it. Ties between equally ranked candidates break in hand-scan order. Table 1 lists the per-policy details needed to read the measurements.

NRU is the only policy that reads dirty state when it picks a victim, though the others can still shift writeback counts indirectly, by leaving pages resident on different trajectories. NRU is not a blanket clean-first rule: an unreferenced dirty page ranks one and is evicted ahead of a referenced clean page at rank two. Aging approximates LRU by sampling rather than tracking it exactly, so a new page can be chosen again before its first sample is even taken. The Working Set timestamps here come from observations made during replacement scans, not from exact last-access times, which makes this an approximate window policy rather than a full implementation of Denning's working-set model or its load control [3]. We run these rules as the project wrote them and do not quietly swap in textbook versions.

Table 1: The six project implementations. A policy name does not imply equivalence to every variant in the literature.

Policy	Decision rule and state update
FIFO	Evict the earliest-loaded resident page. Hits do not update load order; timestamps preserve order when exits leave holes.
Random	On a full-pool miss, use the next <code>std::mt19937</code> output modulo capacity. Hits and free-frame loads consume no random values.
Clock	Scan from the hand, clearing reference bits of one, and evict the first frame with reference bit zero.
NRU	Minimize $2R + D$, where R and D are reference and dirty bits. Periodically reset reference bits before the current access.
Aging	Periodically right-shift a 32-bit counter, insert the old reference bit at the high bit, and clear it. Evict the minimum age; new pages start at zero.
Working Set	Update sampled timestamps and clear reference bits only during victim scans. Evict the first unreferenced page older than the window; otherwise evict the oldest sampled timestamp.

4 Research Theme and Related Work

4.1 From Fault Counts to Load and Writeback Cost

Storage-aware caching studies sources with unequal costs and, in doing so, separates the number of misses from their total cost [4]. Writeback-aware caching goes further and folds loads and writebacks into one cost [2]; its full-lifecycle model assumes an empty cache at both ends of the run. Our measurement window does not share that boundary, so alongside the windowed cost we also report a full-run cost and a hypothetical terminal-drain cost rather than pretending the three coincide. Being optimal for fault count is not the same as being optimal once writebacks are weighted in, and we implement neither OPT nor a weighted optimal lower bound.

CFLRU prefers clean pages inside a candidate region of an LRU list, trading faults against writebacks for flash [9]. WATT keeps access and write timestamps apart and weighs them when it values a page [7]. Both are reasons to separate write locality from the aggregate write ratio, which is what the workloads below do. We do not implement either policy, borrow their hardware constants, or reproduce their throughput numbers. Our weight λ is applied after the fact, offline, and no policy under test ever sees it.

4.2 Working Sets, History, and Scan Resistance

Denning’s working-set model is the standard starting point for reasoning about shifting locality and resident demand [3]. CLOCK-Pro keeps a bounded eviction history to sharpen Clock’s locality decisions and tests it in a VM setting [5], while CAR and ARC adapt to balance recency against frequency [1, 6]. These ideas are why we include scan and hotspot-shift workloads. They are not, however, implemented here: results for ordinary Clock say nothing about CLOCK-Pro or CAR.

More recently, S3-FIFO stacks several queues to absorb short-lived, low-reuse objects, a reminder that workload heterogeneity drives design [8]. That is a different animal from the single FIFO queue tested here, and we make no claim about its concurrent throughput. The references were picked for their bearing on this problem and model.

5 Experimental Design and Data Processing

5.1 Factors and Coverage

The frozen dataset is called `study-v1`. A main trace is 4,096 accesses long: 512 for warmup and 3,584 measured. Warmup leaves frames, dirty bits, the hand, and the sampling clock untouched when measurement begins. Capacity ranges over $F \in \{8, 16, 32\}$ and the write ratio over $q \in \{0, 0.1, 0.5, 0.9, 1\}$. The main study pins the NRU/Aging sampling interval and the Working Set window at four accesses. Input seeds run 0–19 and Random policy seeds 0–4. Every tested capacity divides 2^{32} , so the Random rule does not pick up modulo-reduction bias. Table 2 lays out the factors and the separate sensitivity studies.

Table 2: Experimental factors and sensitivity studies.

Factor	Settings
Workload	Hotspot, scan, shift, and cycle; page sets and phase boundaries are specified below.
Read/write ratio	Read:write ratios 100:0, 90:10, 50:50, 10:90, and 0:100.
Write placement	Uniform, hot-biased, and cold-biased for intermediate ratios on the first three workloads; endpoints and cycle use uniform placement only.
Policy/capacity	Six existing implementations with 8, 16, or 32 frames; a separate 64-frame boundary check.
Cost weight	$\lambda \in \{1, 2, 5, 10, 20\}$, computed offline from each run’s counters.
Replication	Twenty input seeds in the main study; Random uses five policy seeds within each input before paired comparison.
Parameter study	First three workloads, 16 frames, three intermediate ratios, and three placements. NRU/Aging intervals: 1, 4, 16, 64, 256. Working Set windows: 4, 16, 64, 256, 1024.
Length study	First three workloads, 16 frames, intermediate ratios, uniform writes; 1024, 2048, 4096 accesses with 1/8 warmup and proportionally scaled phases.
VM validation	Three capacities $\times$ six policies $\times$ five address-space/lifecycle cases, for 90 cases.

5.2 Synthetic Workloads and Write Labels

Hotspot. Pages run 0–63, and pages 0–15 are the hot set. In each phase, roughly 80% of the positions go to the hot set and the rest to the other 48 pages; the positions are shuffled, and a page is then drawn uniformly from whichever set its position was assigned.

Scan. The scan opens with the hotspot pattern still in force. Between accesses 1536 and 2304 it walks pages 16–63 in order, sixteen times over, after which the hotspot pattern returns. This is a repeated sweep of the non-hot region, not a one-touch-per-page stream.

Shift. Here the hot set is pages 0–15 up to access 2048 and pages 32–47 after it. Both phases keep the roughly 80% hot-access share, with the rest drawn from the other 48 pages.

Cycle. The cycle repeats pages 0–23 with no hot/cold split. Its 24-page working set overflows 8 or 16 frames but fits in 32, which is exactly where capacity thresholds and periodic access start to interact.

Page generation and write-label generation draw from separate random namespaces. Within a replicate, the page order is frozen across algorithms, capacities, write ratios, and placements. Warmup and measurement each get a write budget of $\lfloor qN + 0.5 \rfloor$; over 3,584 measured accesses the intermediate ratios yield 358, 1,792, and 3,226 writes. Hot-biased placement weights current-hot positions nine times as heavily as the rest, and cold-biased placement flips that. Positions are drawn without replacement by scoring each with $\log(1 - u)/w$ and keeping the highest scores up to the budget, where u is an independent uniform draw and w the position weight. Bias is not exclusivity: a large budget has to spill into both categories. The point of the scheme is to change how writes correlate with reuse while holding the page order and the total write count fixed.

5.3 Metrics and Cost Boundaries

Write N for the number of accesses, H for hits, M for faults on valid pages, E for evictions that actually had a victim, and B for dirty-eviction writebacks. Hit rate is H/N and fault rate M/N , and in this model they are complements. Read and write fault rates are reported separately, conditioned on their category; a category with no accesses has an undefined rate, not a zero one. We also track writebacks per thousand accesses ($1000B/N$), the dirty-eviction fraction B/E , dirty occupancy, and the share of the current 16-page hot set that is resident. B/E has no meaning without evictions, and hotspot residency has none for the cycle workload.

We set the page-load cost to one and the writeback cost to λ . The three cost boundaries are

$$C_{\text{measure}}(\lambda) = M_{\text{measure}} + \lambda B_{\text{measure}}, \quad (2)$$

$$C_{\text{full}}(\lambda) = M_{\text{full}} + \lambda B_{\text{full}}, \quad (3)$$

$$C_{\text{full+drain}}(\lambda) = M_{\text{full}} + \lambda(B_{\text{full}} + D_{\text{end}}), \quad (4)$$

where D_{end} counts the dirty pages still resident when the run ends. Equation (4) supposes every one of those pages must be written back; it is an offline sensitivity figure, not an I/O the experiment performs, and not what anonymous-page exit actually does. The weight $\lambda = 5$ is simply one point on the grid, chosen to report against, and not a measured SSD constant. The core study charges one unit per fault, while the VM cases keep zero-fill, swap-in, and file-in counts apart without attaching latencies to them.

For a policy A against Clock, let ΔM and ΔB be the paired differences in faults and writebacks. Then

$$\Delta C(\lambda) = \Delta M + \lambda \Delta B. \quad (5)$$

When the two terms have opposite signs and a positive root exists, the costs cross at $\lambda^* = -\Delta M / \Delta B$. We keep these per-input crossings in the data; each one marks where two fixed trajectories trade places, not a policy adjusting to the weight.

5.4 Paired Analysis, Uncertainty, and Phase Statistics

Clock is the baseline, declared before the runs. For each input we first average Random’s counters over its five policy seeds, then pair the result with Clock on that same input. For costs $C_{A,i}$ and $C_{K,i}$ on input i we report

$$\overline{\Delta C} = \frac{1}{n} \sum_i (C_{A,i} - C_{K,i}), \quad \bar{r} = \frac{100}{n} \sum_i \frac{C_{A,i} - C_{K,i}}{C_{K,i}}. \quad (6)$$

The mean per-input relative change $\bar{r}$ need not match the ratio of the two overall means, and it is undefined wherever the baseline cost is zero. Confidence intervals come from 10,000 input-level paired bootstrap resamples under analysis seed 20260915, taking the 2.5th and 97.5th percentiles

of the mean difference. These bound input uncertainty given the synthetic generator. Random’s variation across policy seeds within an input is exported on its own; five policy seeds are not five independent inputs.

Identical inputs are collapsed, which mostly affects the deterministic cycle endpoints. A cell with a single unique input carries no empirical sampling uncertainty, and repeated deterministic copies are not counted as replicates. We do not apply a multiplicity correction across every configuration comparison, so the emphasis falls on effect sizes and conditional intervals rather than blanket significance claims. For phase analysis the measured region is cut into 28 windows of 128 accesses; the scan and shift results report per-window fault rates and current-hotset residency instead of leaning on whole-trace averages alone. Throughout, the main text shows representative slices of a fully exported matrix rather than one aggregate ranking mashed together across unrelated workloads.

5.5 Execution, Correctness, and Reproducibility

The adapter links the unchanged `MemoryMachine`, built as C++17 with `-O2 -Wall -Wextra -pedantic`. Per-access checks verify unique resident page identity, the hit/fault split, dirty-victim counting, and state consistency, and the adapter’s summaries are cross-checked against the standard full-trace output. For every non-NRU policy, changing write labels while holding page order, capacity, policy seed, and parameters fixed must leave the hit/victim trajectory alone; the audit runs 27,090 such checks and all pass. So do the test suites: 62 project tests, six research tests, and four analysis tests, 72 in all, with no skips. Some project tests cover other modules. Together they establish that the implementation and its exports are consistent, which is a weaker claim than external validity on real systems.

Collection finished on September 15, 2026, on macOS 26.5.2 arm64 with Python 3.11.5 and Apple Clang 17.0.0; the analysis ran on NumPy 2.5.3, pandas 2.2.3, and Matplotlib 3.10.8. After global deduplication the main, parameter, length, and boundary datasets hold 1,082 unique inputs and 31,780 unique core runs, surfaced through 36,340 logical observations. Those observations are not all separate executions: a single run can appear in more than one table. The five offline weights turn the runs into 158,900 cost records without launching anything new, and the 90 VM cases stand apart from all of this. A manifest records input, source, and binary hashes, toolchain metadata, and audit outcomes, and every table and figure in the paper is regenerated from the frozen CSV exports, each carrying its input-file fingerprint.

6 Results and Analysis

6.1 Representative Comparison: Faults and Writebacks Together

Table 3 sets all six policies side by side at 16 frames, 50% uniform writes, and $\lambda = 5$. Aging turns in the highest hit rates and the lowest measured costs on hotspot, scan, and shift, coming in 19.08%, 14.27%, and 18.33% under Clock per input. This holds at the default parameters on three synthetic families and should be read as such, not as a claim that Aging wins everywhere.

On hotspot, NRU takes more faults than Clock, 1,627.0 against 1,569.0, but cuts writebacks from 1,060.6 to 872.0, and the cost falls from 6,871.9 to 5,987.0. Shift shows the same shape: NRU’s 1,641.2 faults exceed Clock’s 1,577.5, yet writebacks drop from 1,069.4 to 881.1. So more faults need not mean a higher writeback-weighted cost. Using the unrounded means for the hotspot slice, $\Delta M = 58.05$ and $\Delta B = -188.60$, which puts the crossing of the mean cost lines near $\lambda = 0.308$; across the whole tested $\lambda \geq 1$ grid NRU stays cheaper here. That crossing is a property of the mean counters, and is not the same as averaging the per-input crossings.

Cycle is where the working set and the policy semantics collide. Sixteen frames cannot

Table 3: Representative main results: means, mean per-input relative changes, and paired absolute cost-difference intervals.

Workload	Policy	Hit(%)	M	B	C_5	$\bar{r}(\%)$	95% CI of ΔC_5
Hotspot	FIFO	53.30	1673.7	1154.9	7448.2	+8.39	[548.3, 601.0]
Hotspot	Random	53.29	1674.2	1097.8	7163.0	+4.25	[249.4, 331.9]
Hotspot	Clock	56.22	1569.0	1060.6	6871.9	+0.00	[0.0, 0.0]
Hotspot	NRU	54.60	1627.0	872.0	5987.0	-12.87	[-946.0, -827.5]
Hotspot	Aging	61.97	1362.8	839.5	5560.1	-19.08	[-1365.6, -1258.7]
Hotspot	Working Set	56.29	1566.7	1059.5	6864.4	-0.11	[-30.9, 14.7]
Scan	FIFO	41.93	2081.2	1292.5	8543.9	+5.53	[430.2, 464.7]
Scan	Random	42.93	2045.4	1243.0	8260.2	+2.03	[133.9, 191.9]
Scan	Clock	44.19	2000.4	1219.2	8096.6	+0.00	[0.0, 0.0]
Scan	NRU	44.79	1978.6	1037.4	7165.6	-11.50	[-988.5, -875.0]
Scan	Aging	49.99	1792.2	1029.8	6941.5	-14.27	[-1199.0, -1114.5]
Scan	Working Set	44.27	1997.2	1218.9	8091.7	-0.06	[-24.4, 13.1]
Shift	FIFO	52.96	1685.8	1161.4	7492.8	+8.22	[533.6, 602.9]
Shift	Random	52.86	1689.6	1107.8	7228.5	+4.41	[268.9, 341.7]
Shift	Clock	55.98	1577.5	1069.3	6924.2	+0.00	[0.0, 0.0]
Shift	NRU	54.21	1641.2	881.1	6046.7	-12.66	[-935.0, -822.3]
Shift	Aging	61.49	1380.0	854.9	5654.6	-18.33	[-1304.1, -1234.6]
Shift	Working Set	55.94	1579.0	1071.2	6935.2	+0.16	[-13.6, 35.2]
Cycle	FIFO	0.00	3584.0	1792.8	12548.2	+0.00	[0.0, 0.0]
Cycle	Random	41.30	2103.8	1325.2	8729.5	-30.43	[-3835.2, -3801.7]
Cycle	Clock	0.00	3584.0	1792.8	12548.2	+0.00	[0.0, 0.0]
Cycle	NRU	48.65	1840.3	915.2	6416.6	-48.86	[-6183.7, -6081.3]
Cycle	Aging	54.27	1639.0	819.0	5734.0	-54.30	[-6851.5, -6776.5]
Cycle	Working Set	0.00	3584.0	1792.8	12548.2	+0.00	[0.0, 0.0]

$F = 16$, $q = 0.5$, uniform writes, $\lambda = 5$, and 3,584 measured accesses; each cell has 20 unique inputs. M denotes faults, B dirty-eviction writebacks, and $C_5 = M + 5B$. Random is averaged over five policy seeds within each input. The last column is a 95% CI of the absolute paired difference, not a percentage interval.

hold 24 cyclic pages, and FIFO, Clock, and the Working Set policy record no hits at all, while Aging and NRU reach 54.27% and 48.65%. What lets them hit is partial retention: some pages survive to be reused while others fault every time around. This depends on the exact period, the sampling, and the hand, and does not carry over to application workloads.

6.2 RQ1: Write Ratio and Write Locality

Figure 1 sweeps the write ratio under uniform placement. For every policy but NRU, write labels play no part in choosing a victim, so with the page order fixed the hit-rate curves do not move; writebacks and cost still shift because the dirty state does. NRU’s curve moves because it ranks on dirty state, but that movement is not a monotone gain with write ratio.

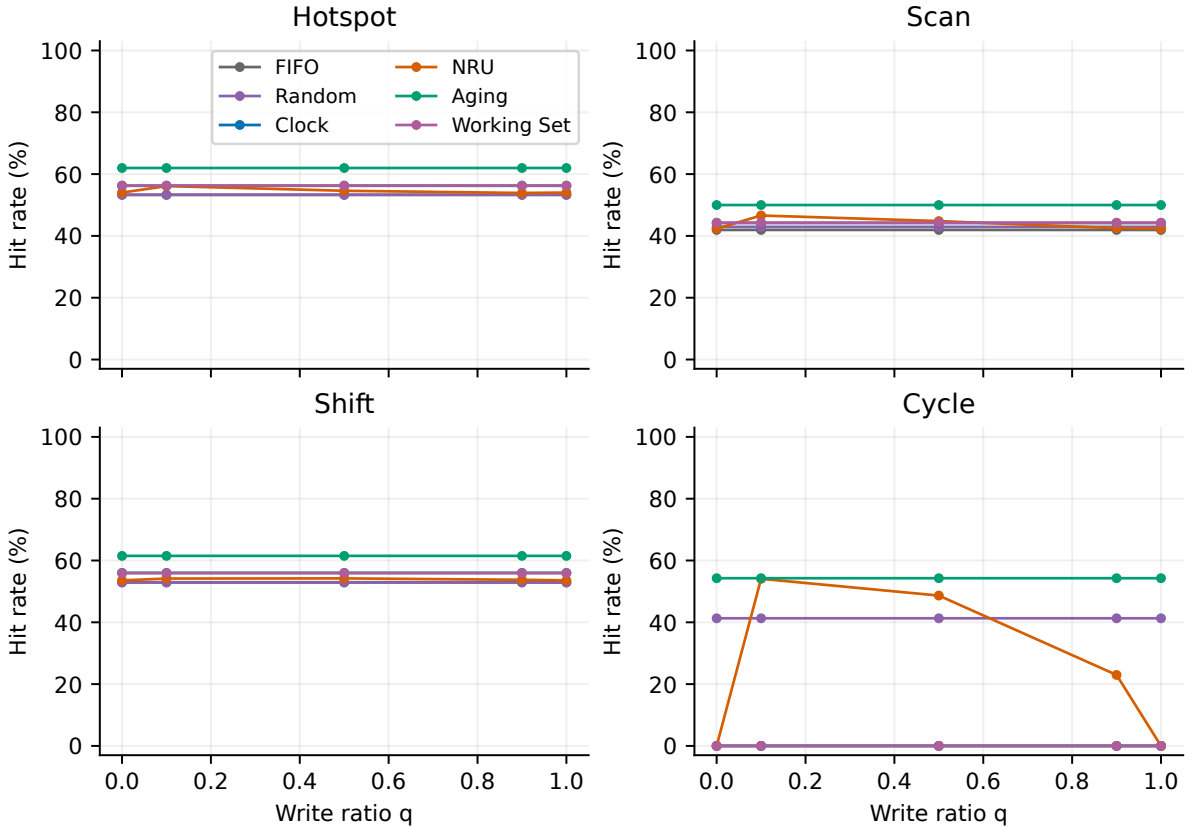


Figure 1: Mean hit rates with 16 frames and uniform writes. Non-NRU residency trajectories are invariant to write labels; overlapping curves indicate equal results.

Table 4 and Figure 2 keep the write budget fixed and move only where the writes land. For hotspot at $q = 0.1$, NRU is 46.81% cheaper than Clock with hot-biased writes and 20.35% cheaper with uniform writes, but 11.38% more expensive with cold-biased writes; the absolute cost-difference intervals are $[-1462.2, -1389.7]$ and $[346.9, 399.3]$ at the two extremes. One aggregate write ratio, in other words, produces opposite cost directions. Shift at the same ratio behaves the same way, going from -42.06% under hot-biased placement to $+11.90\%$ under cold-biased placement.

This fits how NRU blends reference and dirty state. When the pages that get reused are also the ones that get written, the writebacks NRU saves can pay for the extra faults it takes. When the writes instead land on rarely reused pages, holding on to them for their dirty bit just adds load cost. That is the mechanism, not a full causal account, and pinning down individual cases needs the fault and writeback differences together with the full traces. At high write ratios clean victims grow scarce and the edge narrows or reverses: hotspot with uniform $q = 0.9$ writes

Table 4: Write locality can reverse NRU’s cost direction relative to Clock.

Workload	q	Hot-biased(%)	Uniform(%)	Cold-biased(%)
Hotspot	0.1	-46.81	-20.35	+11.38
Hotspot	0.5	-43.09	-12.87	-1.28
Hotspot	0.9	-10.07	+1.84	+3.34
Scan	0.1	-34.37	-16.90	+1.42
Scan	0.5	-34.38	-11.50	-0.40
Scan	0.9	-1.45	+0.99	+1.59
Shift	0.1	-42.06	-16.41	+11.90
Shift	0.5	-42.99	-12.66	-1.15
Shift	0.9	-9.68	+1.61	+3.71

Mean per-input NRU cost change relative to Clock, $F = 16$, $\lambda = 5$. Negative values indicate lower cost. Page order and write count remain fixed.

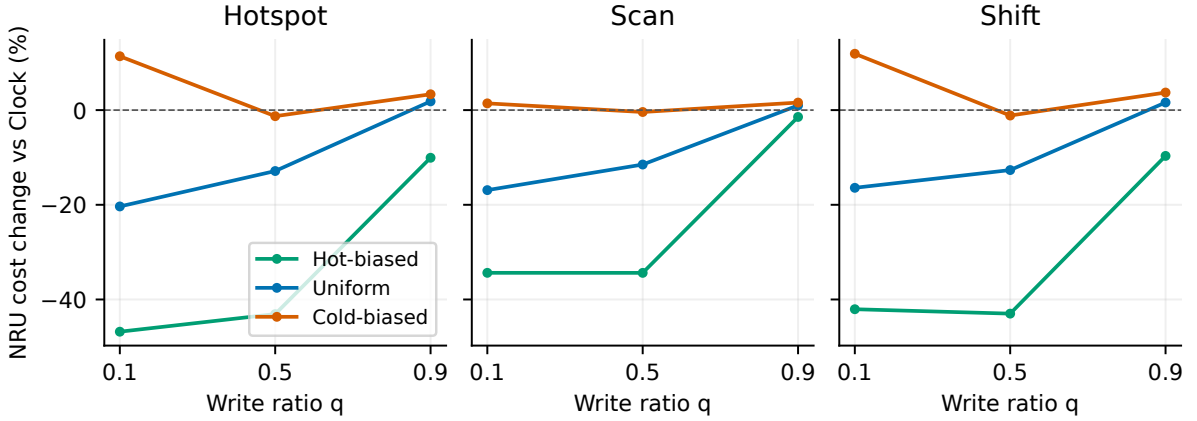


Figure 2: Mean per-input NRU cost change relative to Clock, with 16 frames and $\lambda = 5$. The dashed line denotes equal cost.

costs 1.84% more under NRU than Clock, and cold-biased placement 3.34% more. More writes, then, do not automatically make a dirty-aware ranking pay off.

6.3 RQ2: Resident Capacity and Cost Weight

Figure 3 plots cost per measured access at 8, 16, and 32 frames. Once cycle’s 24-page working set fits in 32 frames, it takes zero measurement faults and zero writebacks after warmup, and the policies become indistinguishable. The 64-frame boundary check likewise holds the full 64-page set of the first three families and removes victim evictions entirely. Even so, a page never touched during a finite warmup still faults on its first load: hotspot, scan, and shift show six, eight, and five such faults, with no writebacks. Enough capacity, that is, does not buy zero faults after a finite warmup. Cycle’s zero-cost baseline has no percentage improvement to report, since dividing by zero would only mislead, and none of these capacity checks promises a fixed proportional payoff from more memory on workloads we did not test.

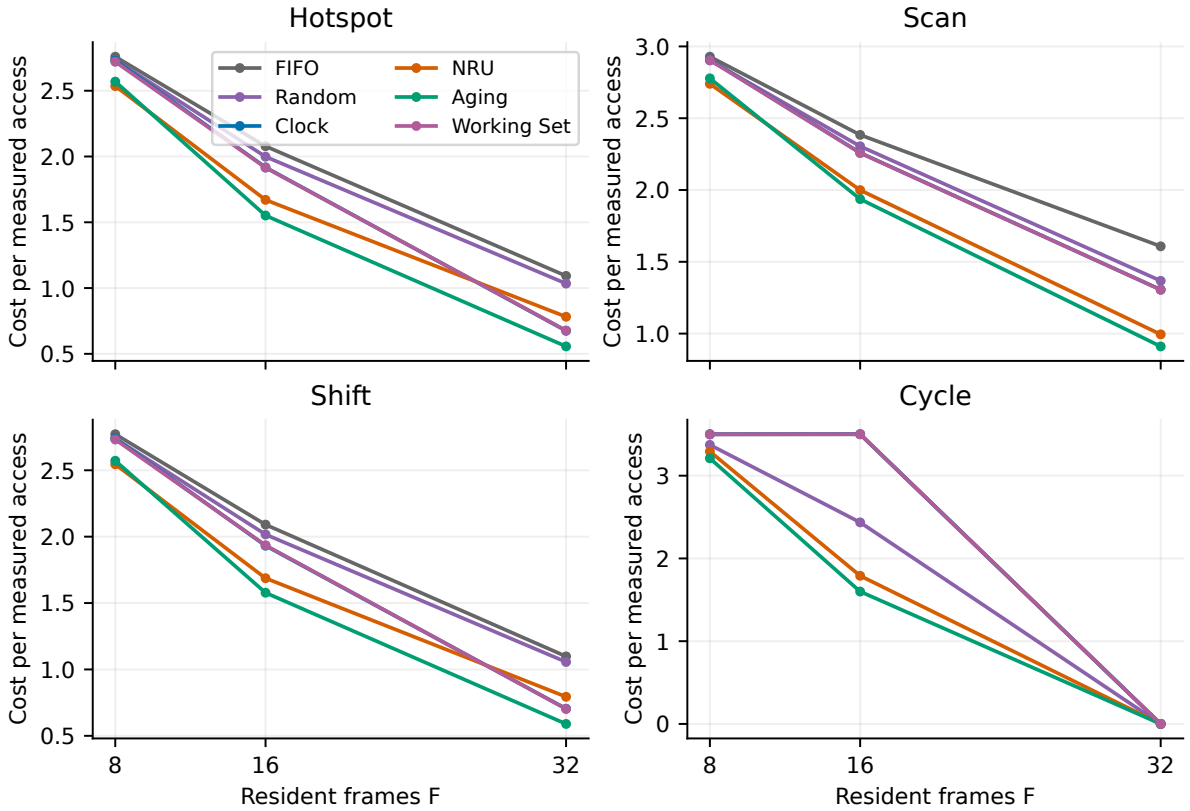


Figure 3: Capacity sensitivity with $q = 0.5$, uniform writes, and $\lambda = 5$. Cost per measured access is $C_{\text{measure}}/3584$.

Figure 4 reweights the same counters under different λ . Each line’s slope is set by its writeback count, not by any policy adapting to λ online, and NRU’s gentler slope than Clock in these slices simply reflects its lower writeback count. A different placement would need its own ΔM and ΔB ; nothing here transfers across slices. The five weights are a sensitivity sweep, not a price list for any particular SSD, swap device, or file system.

6.4 RQ3: Phase Behavior and Parameter Sensitivity

Figure 5 breaks scan and shift into windows. The shaded band is the non-hot sequential scan and the dashed line is the hotspot shift, with residency measured against whichever hot set

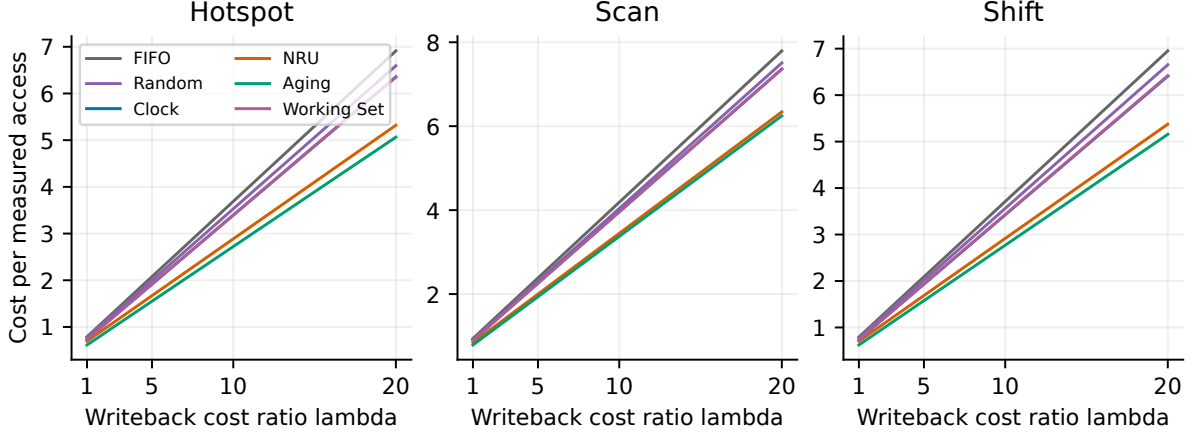


Figure 4: Cost-weight sensitivity with 16 frames and 50% uniform writes. Each curve is an offline reweighting of fixed run counters.

is active at the time. Whole-trace means blur three things together: steady locality, the scan or migration, and the recovery after it. Over the six scan windows Clock faults at 99.86% on average, against NRU’s 88.87% and Aging’s 91.94%. But in the first recovery window Clock faults at 47.66%, below NRU’s 51.80% and Aging’s 52.23%. Leading on aggregate cost, then, does not mean leading in every phase, and the 128-access resolution is too coarse to fix an exact recovery latency or a steady state.

Table 5 gives the grid minima for the same slice, as description only. Aging’s hotspot cost falls from 5,560.1 at the default interval to 3,489.3 at interval 64, while scan and shift bottom out at interval 16. The minimum sits at a different interval for each workload, so no single default speaks for a policy across the board. A longer Aging interval also leaves new pages at age zero until the next sample, which can pile repeated evictions onto a few candidates, so a larger interval is not simply better recency information. Nothing here was tuned on a separate training set, and there is no held-out validation, so these minima are descriptive, not the validated wins of an adaptive policy.

Table 5: Parameter-grid minima for sensitivity analysis, without replacing the default main results.

Workload	Policy	Default cost	Best grid value	Minimum cost
Hotspot	NRU	5987.0	64	5731.3
Hotspot	Aging	5560.1	64	3489.3
Hotspot	Working Set	6864.4	64	6437.6
Scan	NRU	7165.6	64	7162.7
Scan	Aging	6941.5	16	5843.8
Scan	Working Set	8091.7	64	7781.9
Shift	NRU	6046.7	64	5776.4
Shift	Aging	5654.6	16	4611.3
Shift	Working Set	6935.2	64	6527.8

Parameter: sampling interval for NRU/Aging, window for Working Set, in accesses. Descriptive grid minima, without held-out validation.

6.5 RQ4: Finite Boundaries and VM Integration

Table 6 lines up the measurement, full-run, and full-run-plus-drain costs. NRU keeps the same cost direction against Clock across these configurations even as the magnitude moves: for hotspot

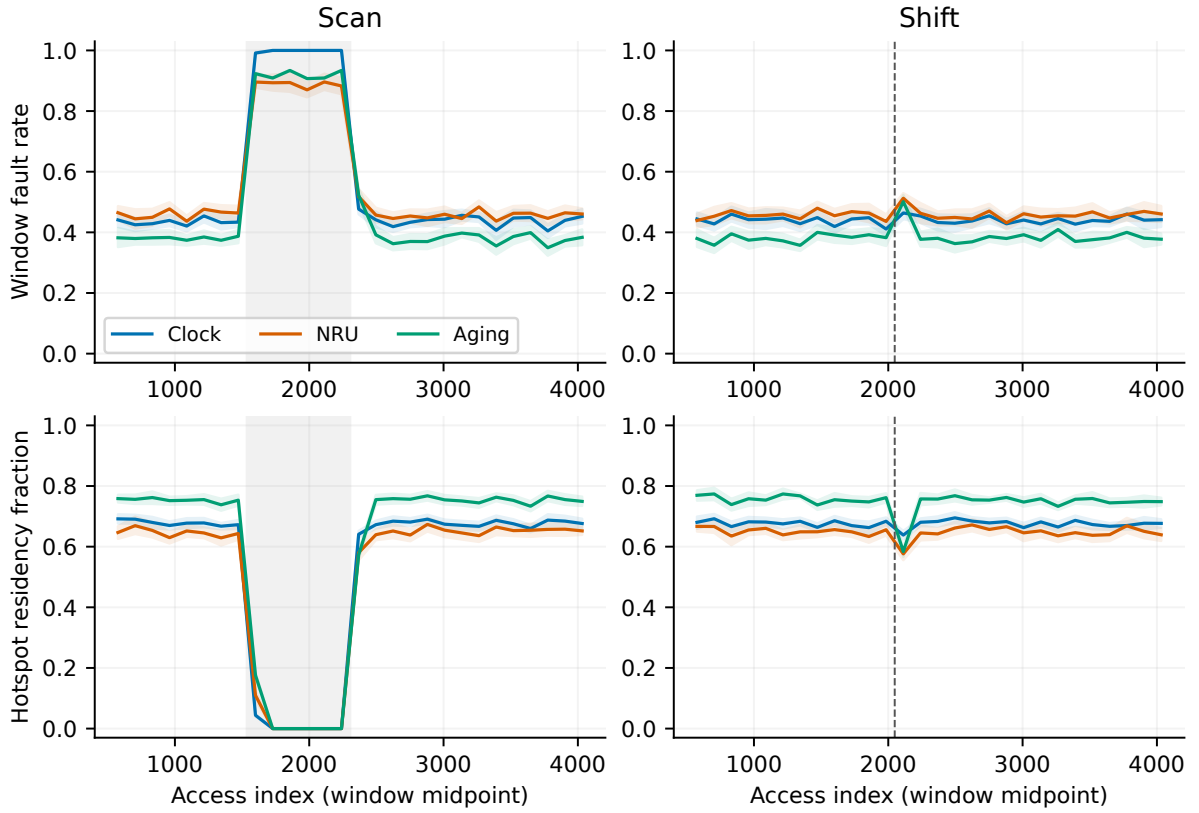


Figure 5: Window fault rates and current-hotset residency, with 16 frames and 50% uniform writes. Shading around curves denotes 95% bootstrap intervals for window means, without cross-window multiplicity correction.

the three numbers are -12.87% , -13.22% , and -12.86% . That is the reason to report several boundaries for a slice, rather than assume any one of them speaks for the others. Figure 6 compares traces of 1,024, 2,048, and 4,096 accesses, each freshly generated at its length rather than cut as a prefix of a longer run. Even the longest is too short to reach a long-run steady state.

Table 6: Endpoint sensitivity: representative NRU/Clock cost changes.

Workload	Measurement(%)	Full run(%)	Full + drain(%)
Hotspot	-12.87	-13.22	-12.86
Scan	-11.50	-11.83	-11.53
Shift	-12.66	-13.19	-12.80
Cycle	-48.86	-48.92	-48.54

$F = 16$, $q = 0.5$, uniform writes, $\lambda = 5$; NRU relative to Clock. Draining is an offline assumption, not anonymous-page exit behavior.

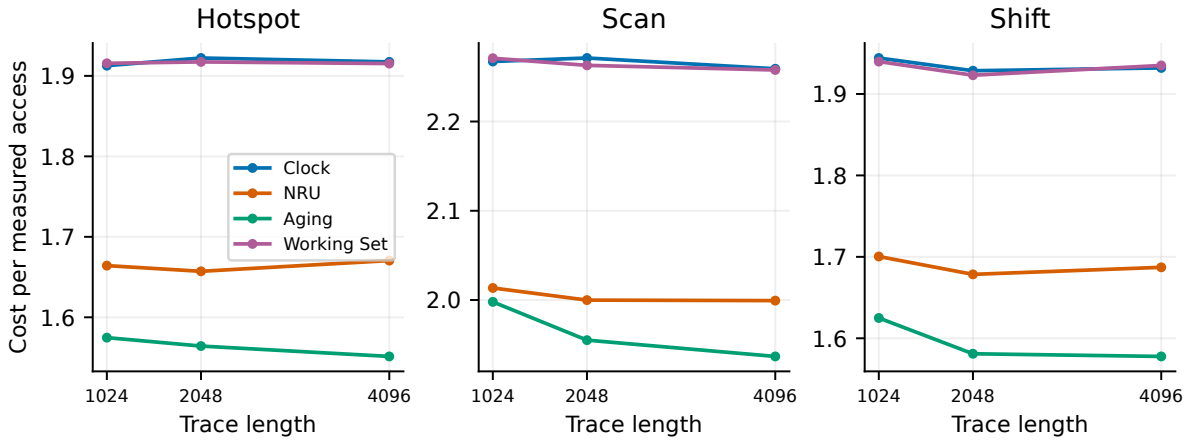


Figure 6: Length sensitivity with 16 frames, 50% uniform writes, and $\lambda = 5$. Warmup occupies one eighth of each trace.

The VM input runs 64 opening hotspot accesses, two sequential scans of pages 16–63, and 32 more hotspot accesses, for 192 valid reads and writes. The hotspot pieces come from the scan generator at input seed zero, writes are placed uniformly at $q = 0.5$, and every page is touched at byte offset 17. The five cases are anonymous, file-mapped, anonymous with exit, file-mapped with exit, and alternating two-process access, with the exit cases adding one instruction. Table 7 shows the semantics under NRU with 16 frames. The single-process anonymous and file cases both hit 147 faults, but for different reasons: 96 zero fills and 51 swap-ins for anonymous pages, against 147 file-ins for mapped ones. Anonymous exit leaves the 62 swap-outs unchanged, while file exit pushes file-outs from 62 to 78, the 16 dirty file pages resident at exit. So the same replacement trajectory can carry different lifecycle I/O, and Equation (4) cannot be reused as an anonymous-exit cost.

The two-process case flips process identity on every access, gives each process a private page table over the shared frame pool, and so works a larger global page set than the single-process case. Its differences come from capacity competition and address-space separation, not from any isolated context-switch cost. The 90 VM cases are integration checks; they do not add independent samples to the main comparisons.

Table 7: Representative VM lifecycle and address-space counters.

Case	Faults	Zero	Swap-in	Swap-out	File-in	File-out	Resident
anonymous	147	96	51	62	0	0	16
file	147	0	0	0	147	62	16
anonymous-exit	147	96	51	62	0	0	0
file-exit	147	0	0	0	147	78	0
two-process	170	117	53	73	0	0	16

NRU, $F = 16$. Each case has 192 valid accesses; exit cases add one exit instruction. The two-process global page set is larger; context-switch overhead is not isolated.

6.6 Trace Cases and Complete Data

Using the predeclared 16-frame, $\lambda = 5$ rule, we pick the inputs at the minimum, maximum, and median of the NRU-minus-Clock absolute cost difference and keep full traces for both policies. The minimum is cycle with 50% uniform writes at seed zero ($\Delta C = -6361$), the maximum is shift with all writes at seed 17 ($\Delta C = 708$), and the median is scan with 50% cold-biased writes at seed eight ($\Delta C = -160$). These are worked examples, and keeping the unfavorable one in view is the point, not a second helping of evidence for average benefit. All of it is retained with the project: the raw counters, all five weights, per-configuration intervals, per-input crossings, window data, and Random’s policy-seed variation.

7 Discussion and Threats to Validity

Scope of the observations. What the data support is looking at faults and writebacks together, and pulling write locality apart from the aggregate write ratio, within this project. What they do not support is a general verdict that NRU beats Clock, or that any one policy is best. The slices shown explain mechanisms; checking other settings still needs the full exported matrix. And the sampling order, the tie rules, and how new pages are initialized are part of the experimental object, so they have to travel with the results.

Distance from hardware. The model leaves out real page contents, background cleaning, asynchronous or parallel I/O, storage write amplification, the finer points of file-system behavior, and CPU caches. The cost C does not count the work of searching for a victim; the exported reference-bit clearing counts hint at scan work but do not fully measure it. Collection wall time is not a controlled throughput benchmark either. For all these reasons the cost stays in normalized page-load units and cannot be read as milliseconds, energy, or device lifetime.

Synthetic workloads and statistical boundaries. The 80% hot-access share, the ninefold write weights, and the phase lengths are design choices, not fits to any real application distribution. Requests do not react to simulated I/O delay, which keeps the experiment controlled but drops the feedback between an application and its storage. A single generator, short traces, fixed defaults, a finite cost grid, and uncorrected multiple comparisons all cap how far the results extrapolate, and no amount of bootstrapping removes limits of that kind. Collapsing deterministic inputs keeps the sample counts honest, but it does not make the workload mix stand in for applications we never ran.

Position relative to prior work. Trading faults against writeback cost is well-trodden ground [2, 7, 9]. This study implements none of CFLRU, WATT, CLOCK-Pro, CAR, ARC,

or S3-FIFO, and claims neither to beat them nor to be the first to write down a writeback-aware objective. For a personal project, the value on offer is the platform itself, the controlled design, the auditable data, and explanations tied to the mechanism. A natural next step keeps the single frame pool but adds faithful reproductions of the literature policies, real traces, held-out parameter validation, or a genuinely new decision rule with a contribution stated up front. Those are not done here, and the comparisons in this paper do not, by themselves, make a strong novelty claim.

8 Conclusion

Beginning from the virtual memory design of Systems Programming Suite, this paper measures the shared replacement core across read/write ratios, write locality, capacity, sampling parameters, and lifecycle boundaries. Taking fewer faults does not guarantee a lower writeback-weighted cost. At one write ratio, how writes line up with reuse can flip NRU’s cost direction relative to Clock, and capacity thresholds, periodic sampling, and termination semantics all shade the reading further. The takeaway for simulator studies is to report the implementation rules, the raw fault and writeback counters, the paired uncertainty, and the boundary sensitivity, not just a hit rate or one overall ranking. Every finding here rests on the frozen **study-v1** dataset and stays inside its tested scope, and the project and its audit pipeline are meant as a footing for deeper systems experiments.

References

- [1] Sorav Bansal and Dharmendra S. Modha. CAR: Clock with adaptive replacement. In *USENIX Conference on File and Storage Technologies (FAST)*, pages 187–200. USENIX Association, 2004. URL <https://cs.stanford.edu/~sbansal/pubs/fast04.pdf>.
- [2] Nathan Beckmann, Phillip B. Gibbons, Bernhard Haeupler, and Charles McGuffey. Writeback-aware caching. In *Symposium on Algorithmic Principles of Computer Systems (APOCS)*, pages 1–15. SIAM, 2020. doi:10.1137/1.9781611976021.1. URL <https://www.cs.cmu.edu/~beckmann/publications/papers/2020.apocs.writeback.pdf>.
- [3] Peter J. Denning. The working set model for program behavior. *Communications of the ACM*, 11(5):323–333, 1968. doi:10.1145/363095.363141. URL https://denninginstitute.com/pjd/PUBS/WSModel_1968.pdf.
- [4] Brian C. Forney, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. Storage-aware caching: Revisiting caching for heterogeneous storage systems. In *USENIX Conference on File and Storage Technologies (FAST)*, pages 61–74. USENIX Association, 2002. URL https://www.usenix.org/legacy/events/fast02/full_papers/forney/forney_html/index.html.
- [5] Song Jiang, Feng Chen, and Xiaodong Zhang. CLOCK-Pro: An effective improvement of the CLOCK replacement. In *USENIX Annual Technical Conference*, pages 323–336. USENIX Association, 2005. URL <https://www.usenix.org/conference/2005-usenix-annual-technical-conference/clock-pro-effective-improvement-clock-replacement>.
- [6] Nimrod Megiddo and Dharmendra S. Modha. ARC: A self-tuning, low overhead replacement cache. In *USENIX Conference on File and Storage Technologies (FAST)*, pages 115–130. USENIX Association, 2003. URL https://www.usenix.org/legacy/events/fast03/tech/full_papers/megiddo/megiddo.pdf.
- [7] Demian Vöhringer and Viktor Leis. Write-aware timestamp tracking: Effective and efficient page replacement for modern hardware. *Proceedings of the VLDB Endowment*, 16(11):3323–3334, 2023. doi:10.14778/3611479.3611529. URL <https://www.vldb.org/pvldb/vol16/p3323-vohringer.pdf>.
- [8] Juncheng Yang, Yazhuo Zhang, Ziyue Qiu, Yao Yue, and K. V. Rashmi. FIFO queues are all you need for cache eviction. In *ACM Symposium on Operating Systems Principles (SOSP)*. ACM, 2023. doi:10.1145/3600006.3613147. URL https://www.pdl.cmu.edu/ftp/Storage/FIFOqueues-SOSP23_abs.shtml.
- [9] Seon-yeong Park, Dawoon Jung, Jeong-uk Kang, Jin-soo Kim, and Joonwon Lee. CFLRU: A replacement algorithm for flash memory. In *International Conference on Compilers, Architecture and Synthesis for Embedded Systems (CASES)*, pages 234–241. ACM, 2006. doi:10.1145/1176760.1176789. URL <https://www.cecs.uci.edu/~papers/esweek06/cases/p234.pdf>.